

LYTESNAP

UNADDICTION ENGINE

Technical Roadmap & System Architecture

RL-Powered Feed Manipulation via Vision-Language-Action Models

CONFIDENTIAL | February 2026

Prepared by Jason & Sage

v3.0 — Deployment Architecture, Competitive Analysis, Product Framing, Self-Adapting Memory

EXECUTIVE SUMMARY

LYTESNAP is an algorithmic autonomy tool that gives users control over social media recommendation algorithms.

Today, platforms like TikTok, YouTube, and Instagram have 100% control over what users see; users have 0%.

LYTESNAP provides a lever. The system deploys a Vision-Language-Action (VLA) agent on a user's physical phone via USB, where it interacts with social media apps as a human would — scrolling, watching videos, and navigating — but with a learned strategy optimized to reshape the recommendation engine toward user-selected topics.

The core innovation is a VAGEN-style world modeling approach where the agent explicitly reasons about the current feed state and predicts how its actions will influence the recommendation algorithm. Trained via PPO on a physical phone farm with rotating target topics, the agent learns a general-purpose feed manipulation strategy that transfers across users and content domains.

No product like this exists. Every parental control app on the market either blocks apps or monitors content. None touch the recommendation algorithm. Parents are already manually co-scrolling with their kids to retrain feeds, but the process is tedious and unsustainable. LYTESNAP automates what parents are already trying to do by hand — and does it with a learned strategy that improves over time.

Product Framing: Algorithmic Autonomy

LYTESNAP does not promise to permanently fix a user's feed. It delivers a capability: the ability to shape the recommendation algorithm on demand. If a teenager manually retrains their feed back to the content the parent reshaped away from, they are making a conscious choice — not being

passively manipulated by an algorithm. That is a better outcome than the status quo even if the content stays the same.

This framing has several advantages. It sets deliverable expectations (a tool, not a cure), reduces liability (no promised outcomes), and opens the market beyond parents: adults trapped in doom-scrolling rabbit holes, people recovering from political radicalization, anyone who wants to reshape their own feed but doesn't have the time or discipline to do it manually. The parent use case is the beachhead, but algorithmic autonomy is the real product.

Users select from a curated list of predefined target topics (e.g., Science & Nature, Arts & Music, Sports, Cooking, History, Technology, Outdoor Activities). The curated list eliminates content taxonomy ambiguity and ensures the agent always targets well-defined content categories. Users pick the direction; the agent handles execution.

COMPETITIVE LANDSCAPE

There are no automated algorithm manipulation products on the market. The competitive landscape falls into three categories, none of which address the core problem LYTESNAP solves.

Parental Control Apps

Products like Bark, Qustodio, Net Nanny, Mobicip, and Kidslox monitor content, block apps, limit screen time, and filter keywords. None interact with the recommendation algorithm. They are bouncers at the door, not interior decorators. A child using TikTok within their allowed screen time still receives the same algorithmically-optimized addictive feed.

Manual Algorithm Reset Guides

ParentsTogether recommends parents manually clear watch history, perform 3–5 parent-approved searches, watch safe videos, and repeat monthly. Verizon published a five-step “retrain” plan urging families to intentionally like, comment, and share preferred content. A grassroots movement of parents manually co-scrolls with their children to hit “Not Interested” on harmful content and like educational videos. These approaches work but are tedious, time-consuming, and parents don’t stick with them. LYTESNAP automates this exact process.

Legislative Approaches

New York and California are passing laws to let parents opt children out of algorithmic feeds, limiting them to chronological posts from followed accounts. California banned algorithms on child accounts without parental approval, with full effect in 2027. These are opt-out solutions that eliminate personalization entirely rather than reshaping it.

The Gap

ParentsTogether research found that test accounts pretending to be 9- and 14-year-olds watching only Roblox gameplay were served 1,325 videos about real guns, school shootings, and weapon tutorials within 30 days. Parents are desperate for a solution. Every existing product says “block the app” or “do it manually.” Nobody automates the manual process. LYTESNAP would be first to market.

CORE ARCHITECTURE

Agent Backbone: Qwen2-VL (3B/7B) with VAGEN-style structured reasoning — StateEstimation via <observation> tokens, TransitionModeling via <prediction> tokens, discrete action output.

RL Algorithm: PPO with Bi-Level GAE. Turn-level advantages for reasoning-action pairs, token-level propagation for individual reasoning quality.

Reward Signal: XML dump at 5-minute checkpoints → LLM extracts and classifies video titles → topic match percentage delta as primary reward, state estimation accuracy as world modeling reward.

Deployment: Desktop app → user connects phone via USB → agent controls the real device → VLA inference runs locally or server-side → actions stream back to phone.

Training: Physical phone farm or emulator-based environment, rotating target topics for domain randomization, parallel trajectory collection feeding centralized PPO updates.

Action Cadence

Social media apps are consumption-first interfaces, but the action cadence varies significantly by content format. Short-form content (TikTok For You Page, YouTube Shorts, Instagram Reels) cycles through videos every 15–30 seconds, requiring the agent to decide every 5–10 seconds whether to keep watching, scroll past, or interact. Long-form YouTube content allows 30–90 seconds between decisions. The agent must handle both cadences: roughly 180–360 decisions in a 30-minute short-form session, or 30–60 decisions in a long-form session. This means inference must complete within 2–3 seconds to stay within the 5–10 second decision window for the

demanding short-form case. A 3B model on llama.cpp can achieve this on modern hardware, but there is no room for leisurely multi-second overhead. The architecture must be optimized for the short-form worst case.

DEPLOYMENT ARCHITECTURE

LYTESNAP supports every user through three local deployment paths. Everything runs over USB on the user's own computer. No cloud device farm, no credential trust issues, no third-party infrastructure dependencies.

Deployment Matrix

User Has	Control Method	Technology
iPhone + Mac	WebDriverAgent over USB/WiFi	Native Xcode toolchain, pymobiledevice3
iPhone + Windows	WebDriverAgent via go-ios	go-ios binary, iTunes for USB drivers
Android + Any Computer	Raw adb	Android Debug Bridge over USB

Path 1: iPhone + Mac (Native WDA)

WebDriverAgent (WDA) is a Facebook-developed HTTP server that runs on an iPhone as a persistent XCUITest test case, exposing native iOS automation functions as REST API endpoints. The LYTESNAP Mac app bundles a pre-signed WDA IPA (signed with our \$99/year Apple Developer certificate). When the user plugs in their kid's iPhone, the app automatically installs WDA (invisible to the user), launches it, and the agent sends HTTP commands to control the device: tap, swipe, screenshot, element queries.

WiFi automation is available after the initial USB pairing, enabling the parent to start a session without keeping the phone tethered. WDA expires annually with our paid developer certificate, refreshed automatically via app updates.

Path 2: iPhone + Windows (go-ios)

go-ios is a Go-based, operating-system-independent implementation of iOS device features that runs on Windows, Linux, and Mac without requiring Xcode. It is production-proven: HeadSpin.io and Sauce Labs (major device farm companies) use it in their infrastructure. The LYTESNAP Windows app bundles the go-ios static binary (single executable, no dependencies beyond iTunes for Apple USB drivers). The deployment chain is identical to the Mac path: install pre-signed WDA, launch WDA HTTP server, agent sends commands to WDA REST API.

go-ios uses private Apple APIs that can break with iOS updates. This is a manageable risk for LYTESNAP because sessions are 30–60 minutes weekly (not 24/7 CI), we can ship updates within days if iOS breaks compatibility, and the go-ios maintainer actively tracks iOS releases.

Path 3: Android + Any Computer (Raw adb)

Android devices are controlled via raw Android Debug Bridge (adb) shell commands over USB. This is the simplest and most robust path: zero on-device footprint, no session state, no framework to maintain. The agent sends adb commands for tap coordinates, swipe gestures, text input, and screenshot capture. No Appium, no UiAutomator2 — direct shell interaction with the device. Works identically on Mac, Windows, and Linux.

Build Pipeline

A single Mac in our CI pipeline handles the one-time WDA build and code signing. This Mac is never user-facing — it produces the signed WDA IPA that ships bundled inside both the Mac and Windows desktop apps. The \$99/year Apple Developer Program account is the only recurring cost for iOS support.

Inference Strategy

The 5–10 second decision window for short-form content (TikTok, YouTube Shorts) requires inference to complete within 2–3 seconds. A 3B parameter model on llama.cpp can achieve this on modern machines with a discrete GPU or Apple Silicon, making local inference viable for a significant portion of the user base. This eliminates server costs and solves the privacy concern: screenshots never leave the device. For machines that cannot meet the 2–3 second inference target, server-side inference remains available as a fallback. Long-form YouTube content is more forgiving, with 30–90 seconds between decisions.

Local inference + local device control = fully offline operation for users with capable hardware (discrete GPU or Apple Silicon). Zero server costs, zero privacy exposure, zero internet dependency during sessions. Server fallback handles underpowered machines. The only cloud component is model weight downloads and software updates.

DESIGN PRINCIPLES

- 1. Algorithmic Autonomy:** The product delivers control, not outcomes. Users currently have zero influence over what the recommendation algorithm shows them. LYTESNAP provides a lever. Whether the user maintains the reshaped feed or lets it drift back is their choice — the point is they now have one.
- 2. Judo Philosophy:** Leverage the algorithm's own stickiness. The same persistence that makes feeds addictive ensures that once flipped, the new content profile is maintained by the platform itself.
- 3. Vision-First Agent, Structured Reward:** The agent operates in pixel space for human-like behavior and detection resistance. Reward computation uses XML hierarchy dumps for accuracy. Best of both worlds.
- 4. Interpretable World Model:** VAGEN-style reasoning produces human-readable traces of the agent's understanding and strategy, enabling debugging, trust-building, and user-facing transparency.
- 5. Topic-Agnostic Training:** Rotating target topics during training produces a general feed-manipulation policy, deployable against any user-selected topic from the curated list without retraining.
- 6. Local-First:** All device control and (where possible) inference runs locally on the user's machine. No cloud infrastructure per user session. No screenshot streaming. No credential trust to third parties. The parent plugs in the phone, selects topics, and the agent runs entirely on their hardware.

PHASED ROADMAP

PHASE 0a: YOUTUBE PLAYWRIGHT MVP

Week 1

Ship a working product before investing in ML infrastructure. YouTube’s recommendation engine is unified across web and mobile — reshaping the feed on youtube.com carries over to the YouTube app on any device. A Playwright browser automation script can manipulate the feed without any phone farm, emulator detection risk, or RL training.

Task	Details
Playwright Script	Browser automation that searches target topics from the user’s curated list, watches videos for realistic durations (30–90 seconds with natural pause patterns), hits “Not Interested” on unwanted content, likes and subscribes to healthier channels, and clears watch history for problematic content. Repeat across multiple sessions.
Authentication Flow	The parent manually logs into their Google/YouTube account through the Electron app’s embedded browser. Playwright inherits the authenticated session. This avoids triggering Google’s suspicious login detection (unfamiliar device, different browser fingerprint, 2FA challenges). Session cookie management and expiry handling required.
Session Strategy	Define hardcoded feed-flipping playbook: topic search → watch → engage → suppress. Implement realistic human-like timing with jitter. Run sessions of 30–60 minutes.
Cross-Device Validation	Confirm that YouTube web interactions carry over to the YouTube mobile app on the user’s phone. Test on both iOS and Android. Measure feed composition convergence.
Topic Selection UI	User selects from curated list of predefined target topics (Science & Nature, Arts & Music, Sports, Cooking, History, Technology, Outdoor Activities, etc.). Agent maps selected topics to search queries and channel targets.
Monetization	Ship as paid product while building the RL agent in parallel. Revenue validates market demand and funds development of the full system.

Exit Criteria

- Playwright script achieves measurable feed shift (>20% topic match increase) within 5 sessions
- YouTube web manipulation confirmed to transfer to mobile app on both iOS and Android
- Authentication flow works reliably without triggering Google security challenges
- 10+ paying beta users running the product successfully

Phase 0a is the fastest path to revenue. YouTube is the only major platform where web manipulation transfers to mobile. TikTok, Instagram, and others require direct device control (Phases 0b–5). Ship YouTube MVP in weeks, not months.

Validate that programmatic interaction with TikTok and Instagram on a real phone can measurably shift the recommendation feed. This phase proves the core technical assumptions for platforms that require direct device control.

Task	Details
Phone + adb Setup	Configure a single Android device with USB debugging. Establish raw adb connection. Verify reliable screenshot capture and UI hierarchy extraction from TikTok app via adb shell commands.
Manual Feed Flip Test	Write a hardcoded adb script that executes a simple feed-flipping strategy: navigate to search, enter target topic, watch 3–5 videos for 60 seconds each, return to For You Page. Run daily for one week.
Reward Pipeline Prototype	Build the screenshot → LLM title extraction → topic classification pipeline. Validate accuracy by comparing LLM-classified content against manual human labels on 100+ feed screenshots.
Feed Persistence Measurement	After running the hardcoded agent for 7 days, stop all automated interaction. Measure feed composition daily for the next 14 days. Quantify the decay rate of recommendation changes under both zero activity and normal organic usage.
Action Space Definition	Catalog all actions the agent can take within TikTok mobile via adb: scroll, tap video, watch for N seconds, navigate home, search, type query, back button. Define the discrete action set and hard safety masks (never like, follow, comment, post).
iOS Deployment Test	Test WDA deployment on iPhone via Mac (native) and Windows (go-ios). Verify the same adb-equivalent commands work through WDA REST API. Confirm both deployment paths are production-ready.

Exit Criteria

- adb can reliably control TikTok on a physical phone for 30+ minute sessions without crashes
- Hardcoded script achieves measurable feed shift (>20% topic match increase) within 5 sessions
- Feed persistence confirmed: recommendation changes survive 48+ hours without agent intervention
- Reward pipeline achieves >90% agreement with human labels on topic classification
- WDA deployment validated on both Mac (native) and Windows (go-ios) for iPhone control

Build the parallel training infrastructure and collect the behavioral cloning dataset needed to bootstrap the RL agent with a warm-start policy. Training can begin on emulators for speed and cost efficiency, with validation on physical devices to ensure transfer.

Task	Details
Training Environment	Set up training infrastructure using Android emulators (for speed and parallelism) and/or a small physical phone farm (10 devices, Pixel 3a/4a at \$30–60 each). Emulators allow rapid iteration; physical devices validate that learned policies transfer to real hardware and pass platform detection.
Orchestration Layer	Build a Python orchestrator that manages all training devices/emulators via raw adb, monitors health, and restarts sessions on failure. Each instance runs independently with centralized logging and trajectory collection.
Demonstration Collection	Manually execute feed-flipping strategies across diverse topics from the curated list on all training instances. Record full trajectories: screenshots (every 5 seconds), UI state dumps (every 5 minutes), and human actions with timestamps. Target: 300–500 demonstration trajectories.
Trajectory Storage Pipeline	Design storage format for training data: compressed screenshot sequences, UI state snapshots, action logs, reasoning annotations, reward checkpoints. Build efficient data loading for batched training.
Topic Rotation Protocol	Use the predefined curated topic list as the training curriculum. Build automated topic assignment and rotation across training instances. Include both broad categories (Science) and specific subcategories (Marine Biology) for curriculum diversity.

Exit Criteria

- Training environment operational with automated session management and health monitoring
- 300+ demonstration trajectories collected across 15+ distinct target topics from curated list
- Data pipeline validated: trajectories load efficiently for training, reward labels verified
- Physical device validation confirms emulator-trained behaviors transfer to real phones

Build the VLA agent, train the initial policy via behavioral cloning, and stand up the full PPO training loop with VAGEN-style world modeling.

Task	Details
VLA Backbone Setup	Configure Qwen2-VL (3B) as the base vision-language model. Implement the structured reasoning output format: <observation> tokens for state estimation, <prediction> tokens for transition modeling, <action> tokens for discrete action output.
Behavioral Cloning Warm Start	Fine-tune Qwen2-VL on the demonstration trajectories from Phase 1. The model learns to imitate human feed-flipping behavior including generating plausible reasoning traces. This provides the warm-start policy for PPO.
PPO Training Loop	Implement PPO with Bi-Level GAE. Turn-level advantages evaluate whether each reasoning-action pair was productive. Token-level advantages propagate credit to individual reasoning tokens. Dual reward: 5-minute topic percentage delta (primary) + state estimation accuracy (world modeling).
Inference Pipeline	Build the action loop: phone screenshot → VLA inference → action returned via adb/WDA. Support both local inference (llama.cpp on user's machine) and server-side inference (vLLM). Short-form content (TikTok, Shorts) requires 2–3 second inference to fit within 5–10 second decision windows. Long-form YouTube content is more forgiving.
Safety Action Filter	Implement hard constraints on the action space. Block all engagement actions (like, follow, comment, post), prevent navigation outside target app, block ad interactions and purchases. Actions are filtered before execution regardless of what the policy outputs.
Online RL Training	Run PPO training on the training environment. Each instance collects trajectories autonomously, ships data to training server. Centralized policy updates, new weights pushed back. Monitor training curves: reward, topic match percentage, reasoning quality, action entropy.
Exploration Strategy	Implement entropy bonus scheduling and periodic exploration episodes with elevated temperature. Optionally add curiosity-driven intrinsic reward for discovering novel feed states the agent hasn't encountered before.

Exit Criteria

- BC warm-start policy achieves >30% topic match improvement over random within 30-minute sessions
- PPO training shows monotonic improvement in feed-flip success rate over 2+ weeks
- Agent discovers at least one non-obvious strategy not present in human demonstrations
- Reasoning traces are coherent and accurately reflect feed state >80% of the time
- Safety filter has zero violations across all training episodes
- Local inference achieves <3 second latency per decision on machines with discrete GPU or Apple Silicon

Rigorously evaluate the trained policy across diverse conditions and harden it for real-world deployment.

Task	Details
Cross-Topic Generalization	Evaluate on held-out topics from the curated list never seen during training. Measure feed-flip success rate and compare against in-distribution topics. The agent should generalize to novel topics without retraining.
User Profile Diversity	Test on accounts with diverse watch histories: heavy gaming, beauty/fashion, news, music, mixed. Verify the calibration phase (observation-only episodes) enables adaptation to unfamiliar starting states.
Behavioral Detection Audit	Analyze the agent's interaction patterns for bot-like signatures: timing regularity, scroll speed consistency, watch duration distributions. Compare statistical properties against recordings of real human usage. Tune if needed.
Persistence Validation	Run the agent on test accounts, then cease all interaction. Monitor feed composition at 1, 3, 7, and 14 days. Test under both zero activity and simulated organic usage (adversarial teenager behavior). Quantify decay rates for both scenarios.
A/B Policy Testing	Compare trained RL policy against the Phase 0b hardcoded baseline and the BC-only policy. Measure feed-flip rate, time to target percentage, strategy diversity, and reasoning accuracy across all three.
Continuous Monitoring System	Build automated performance tracking: daily success rate metrics from the training environment, alerting when performance drops below thresholds, triggering retraining pipeline when platform algorithm updates degrade the policy.

Exit Criteria

- Agent achieves >60% topic match on held-out topics within 3 sessions
- No statistically significant difference in interaction patterns vs. human baseline ($p > 0.05$)
- Feed persistence confirmed across 7+ days on 80%+ of test accounts
- RL policy outperforms hardcoded baseline by >2x on feed-flip rate

Build the user-facing desktop application supporting all three deployment paths (iPhone+Mac, iPhone+Windows, Android+Any) in a single unified interface with local-first architecture.

Task	Details
Desktop App (Electron)	Cross-platform application with automatic device detection on USB connection: iPhone detected → WDA path, Android detected → adb path. Guided setup for USB debugging (Android) or trust dialog (iPhone). User selects target topics from curated list, schedules sessions, monitors progress.
Mac Build: Native WDA	Bundle pre-signed WDA IPA. Auto-install WDA on iPhone connection via pymobiledevice3 or libimobiledevice. WiFi automation after initial USB pairing. Seamless experience — user never sees WDA.
Windows Build: go-ios	Bundle go-ios static binary and pre-signed WDA IPA. Require iTunes for Apple USB drivers (guided install if missing). Same WDA deployment chain as Mac, transparent to user.
Android: Raw adb	Bundle platform-appropriate adb binary. Zero additional dependencies. Fastest setup path of the three.
Local Inference Runtime	Bundle llama.cpp or equivalent local inference engine. Auto-detect hardware capabilities (GPU, RAM) and select appropriate model quantization. Must achieve <3 second inference for short-form content cadence. Fall back to server-side inference for machines that cannot meet latency target. Minimum viable hardware: discrete GPU or Apple Silicon with 8GB+ RAM.
Server Inference Fallback	GPU inference servers for users whose hardware cannot run local inference. Screenshot streaming protocol with E2E encryption. Auto-scaling for concurrent sessions. Latency target: <500ms round-trip.
Privacy & Compliance	Local-first architecture means screenshots never leave the device for most users. For server fallback: E2E encryption, zero-retention policy. COPPA compliance review for minor user data. Privacy policy and terms of service. Legal review of CFAA exposure for automated interaction with third-party platforms.

User Onboarding Flow	First-run experience: plug in phone, app detects device type, guided setup, select target topics from curated list, run initial session. Real-time visualization of agent's actions.
Progress Dashboard	Sessions completed, feed composition over time, topic match trend line. Parents can monitor their child's feed transformation progress.
Maintenance Scheduling	Automated periodic sessions (e.g., weekly) to counteract drift from organic usage. Smart scheduling that detects when feed composition has degraded and triggers a maintenance run.

Exit Criteria

- App installs and connects to phone in <5 minutes for non-technical users on all three paths
- Local inference achieves <3 second latency on machines with discrete GPU or Apple Silicon
- Server fallback handles 50+ concurrent sessions with <500ms latency
- 10 beta users successfully run the product for 2+ weeks across Mac, Windows, iPhone, Android
- Legal review completed for CFAA and platform TOS exposure

Expand to additional platforms, implement self-adapting memory, and scale the product.

Task	Details
TikTok Expansion	Adapt the agent to TikTok’s mobile app. Leverage the same VLA backbone with TikTok-specific fine-tuning. The VAGEN reasoning framework transfers directly — only UI grounding needs retraining. TikTok’s aggressive algorithm makes it the highest-impact target.
Instagram Reels	Extend to Instagram’s short-form content. Similar architecture adaptation. Instagram’s Explore feed and Reels algorithm present unique manipulation opportunities distinct from YouTube and TikTok.
Self-Adapting Memory (Titans)	Integrate a Google Titans-style learned memory module into the VLA backbone. The memory updates its own parameters at inference time based on prediction surprise — when the agent’s VAGEN-style <prediction> tokens diverge from observed reality (UI changes, algorithm behavior shifts), the memory module learns the new patterns automatically. This eliminates the need for full retraining when platforms update. The 5–10 second decision window for short-form content is tight for concurrent memory gradient updates; memory updates may need to be batched or run asynchronously during longer watch periods and search navigation. See Future Architecture section for details.
Online Continual Learning	With user consent, collect anonymized trajectory telemetry from deployed agents. Use this data for continuous policy improvement. Implement federated learning or privacy-preserving aggregation to update the policy without accessing raw user data.
Algorithm Adaptation	Build automated detection of platform algorithm changes. When the training environment’s success rate drops below threshold, trigger retraining. The VAGEN reasoning traces provide interpretable signals about what changed. Titans memory module provides immediate short-term adaptation while retraining handles long-term shifts.

Distribution Strategy	Mac app distributed directly from website (Mac App Store likely won't allow it due to TOS concerns with device automation). Windows app via website download. Explore partnerships with school districts and parental advocacy groups.
-----------------------	--

Exit Criteria

- TikTok agent achieves comparable feed-flip rates to YouTube agent
- Titans memory module demonstrates automatic adaptation to at least one platform update without retraining
- Online learning shows measurable improvement over static policy within 30 days of deployment
- Product supports 500+ concurrent users across YouTube and TikTok

FUTURE ARCHITECTURE: SELF-ADAPTING MEMORY

The single biggest operational risk in the roadmap is platform updates breaking the trained agent. Google Titans (2025) offers an elegant solution that maps naturally onto LYTESNAP's existing architecture.

The Problem

DigiRL (Berkeley, 2024) demonstrated that frozen VLM policies degrade as apps update. The current roadmap handles this with continuous monitoring and retraining on the phone farm — but retraining is expensive, reactive (detects degradation after it hurts users), and requires maintaining training infrastructure indefinitely.

The Titans Solution

Google Titans introduces a learned long-term memory module for transformers. The memory is a neural network whose weights are the memory itself — it updates its own parameters at test time via a gradient-based mechanism. Incoming data that causes high prediction loss (surprise) gets memorized; predictable data gets forgotten.

Why It Maps to LYTESNAP

VAGEN already provides the surprise signal. The agent generates <prediction> tokens about what it expects to happen after each action. When a platform changes its UI or algorithm behavior, those predictions start being wrong. That prediction error is the high-loss signal that drives Titans memory updates. No extra machinery needed — the world model's own failure becomes the learning signal.

The action cadence is an important constraint. Short-form content (TikTok, YouTube Shorts) requires decisions every 5–10 seconds, leaving only 2–7 seconds of idle time between inference calls. This is tight for concurrent memory gradient updates. Two approaches: batch memory updates during longer idle periods (search navigation, topic switching, long-form video watches) where the agent has 10–90 seconds of breathing room, or run memory updates asynchronously on a background thread while the main inference loop continues with slightly stale memory. Either approach is tractable but adds engineering complexity beyond the base Titans implementation.

What It Could Adapt To

UI layout changes (button positions, new navigation patterns), algorithm behavior shifts (different response to the same manipulation strategy), per-user variation (A/B tests where different accounts see different UI versions), and platform-specific quirks across YouTube, TikTok, and Instagram. All without retraining the base policy.

Architecture Options

Titans proposes three integration variants: Memory as Context (MAC), Memory as Gate (MAG), and Memory as Layer (MAL). MAG is the most promising for LYTESNAP — it gates the attention output with memory, allowing the base policy to operate normally while the memory modulates its behavior based on accumulated experience. The memory module would need to be added to the Qwen2-VL backbone and trained alongside the base policy, with memory updates frozen during PPO training and active only at deployment.

Open Questions

How does PPO training interact with a module that also updates at inference? How do you prevent memory drift (locking in wrong patterns from misinterpreted observations)? Should the safety action filter double as a drift detector? These are research questions to explore after the core product ships.

Titans is a Phase 5+ upgrade. Ship with retraining-based adaptation first (proven, works today). Explore Titans integration as the “never retrain again” evolution. If it works, it’s a massive competitive moat — the agent self-adapts while any future competitor must manually retrain.

CRITICAL RISKS

Risk	Impact	Mitigation
Legal Exposure (CFAA)	Automating interaction with third-party accounts may raise Computer Fraud and Abuse Act concerns beyond simple TOS violations	Engage attorney before launch. The user authorizes access to their own (or their minor child's) account. Frame as assistive technology, not unauthorized access. Precedent research needed.
Platform TOS	Automated interaction violates platform terms of service, risking account bans	Human-like behavior via RL, rate limiting, real device usage, physical device fingerprints. TOS ban is account-level risk for user, not legal risk for LYTESNAP.
Privacy / COPPA	Processing data from minors' devices creates legal liability	Local-first architecture: screenshots never leave device. Server fallback uses zero-retention processing with E2E encryption. Legal review for COPPA compliance.
YouTube Auth Flow	Google's suspicious login detection blocks automated browser sessions, 2FA challenges	Parent manually authenticates in embedded browser. Playwright inherits session. Cookie management for session persistence across runs.
Inference Latency	Server-side VLA inference adds network round-trip	Local inference via llama.cpp eliminates round-trip entirely. Server fallback for underpowered machines with <500ms target.
Algorithm Updates	Platform algorithm changes invalidate trained policy	Continuous monitoring with automated retraining. Long-term: Titans memory module for automatic self-adaptation.
iOS Compatibility	iOS updates break go-ios and WDA compatibility	Track go-ios releases, ship updates within days. WDA maintained by active Appium community. Mac native path as fallback.
Emulator Detection	TikTok's aggressive anti-bot system shadowbans automated accounts	Physical devices for deployment (always). Training can use emulators with physical device validation. Real device sensors pass all detection checks.

TECHNOLOGY STACK

Component	Technology
Agent Backbone	Qwen2-VL 3B/7B with VAGEN-style structured reasoning
RL Framework	PPO with Bi-Level GAE, implemented via verl or CleanRL
Android Automation	Raw adb shell commands over USB (zero framework dependency)
iPhone Automation (Mac)	WebDriverAgent via native Xcode toolchain / pymobiledevice3
iPhone Automation (Win)	WebDriverAgent via go-ios (Go binary, no Xcode required)
YouTube Automation	Playwright browser automation (Phase 0a MVP)
Local Inference	llama.cpp or equivalent for on-device VLA serving
Server Inference	vLLM or SGLang for high-throughput VLA serving (fallback)
Reward Pipeline	Screenshot/XML → LLM title extraction (Qwen2.5-7B) → topic classification
Training Infra	DGX Spark / GPU server for centralized PPO updates, emulators + physical devices
Desktop App	Electron + React (cross-platform: Mac + Windows)
Data Storage	Trajectory database with compressed images, structured metadata
Privacy	Local-first architecture, E2E encryption for server fallback, zero-retention
Code Signing	\$99/year Apple Developer Program for WDA IPA signing
CI Pipeline	Single Mac for WDA build + signing (not user-facing)
Future: Memory	Google Titans-style learned memory module for self-adaptation

Let's change the fucking world.

REQUIRED READING

The following papers form the theoretical and practical foundation for the LYTESNAP unaddiction engine. They are organized by relevance to the system architecture.

Core Architecture

VAGEN: Reinforcing World Model Reasoning for Multi-Turn VLM Agents

Wang, Zhang, Wang, Gao, Li et al. • arXiv:2510.16907

Why: Primary architectural inspiration. Introduces the POMDP formulation with explicit StateEstimation and TransitionModeling, Bi-Level GAE for turn-aware credit assignment, and World Modeling Reward. A 3B model achieves 3× improvement over untrained baselines.

<https://arxiv.org/abs/2510.16907>

Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution

Wang, Bai, Tan, Wang, Fan et al. (Alibaba) • arXiv:2409.12191

Why: Our agent backbone. Introduces Naive Dynamic Resolution for processing variable-resolution screenshots and M-RoPE for fusing positional information across text/images/video. The 3B/7B variants are our target model sizes.

<https://arxiv.org/abs/2409.12191>

Qwen2.5-VL Technical Report

Bai et al. (Alibaba) • arXiv:2502.13923

Why: Latest iteration of our backbone with Window Attention for reduced compute, dynamic FPS sampling for video, and absolute time encoding. Excels at document parsing and object localization.

<https://arxiv.org/abs/2502.13923>

RL for GUI / Mobile Agents

DigiRL: Training In-The-Wild Device-Control Agents with Autonomous RL

Bai, Zhou, Cemri, Pan, Suhr, Levine, Kumar (UC Berkeley) • arXiv:2406.11896 • NeurIPS 2024

Why: Closest prior work. Trains a 1.3B VLM via offline-to-online RL on Android emulators, achieving 49.5% absolute improvement over SFT. Critical finding: frozen policies degrade as apps update — validating both our continuous retraining design and the Titans self-adaptation path.

<https://arxiv.org/abs/2406.11896>

MobileRL: Online Agentic Reinforcement Learning for Mobile GUI Agents

Xu et al. (ByteDance / Tsinghua) • arXiv:2509.18119

Why: Introduces AdaGRPO with difficulty-adaptive positive replay and failure curriculum filtering. Trains on Qwen2.5-VL-7B using 1000+ Dockerized Android emulators. Their orchestration architecture directly informs our training environment design.

<https://arxiv.org/abs/2509.18119>

MobileGUI-RL: Advancing Mobile GUI Agent through RL in Online Environment

Shi et al. (Google) • arXiv:2507.05720

Why: Adapts GRPO for trajectory-aware advantages with multi-component rewards. Their synthetic task generation pipeline is relevant to our topic rotation curriculum design.

<https://arxiv.org/abs/2507.05720>

UI-TARS: Pioneering Automated GUI Interaction with Native Agents

Qin et al. (ByteDance) • arXiv:2501.12326

Why: SOTA native GUI agent. Screenshot-only perception with dense captioning, unified action modeling, and System-2 reasoning. Their iterative training with reflective online traces is the template for our data flywheel.

<https://arxiv.org/abs/2501.12326>

UI-TARS-2: Advancing GUI Agent with Multi-Turn Reinforcement Learning

Wang et al. (ByteDance) • arXiv:2509.02544

Why: Extends UI-TARS with stabilized multi-turn RL and data flywheel. Achieves 47.5 on OSWorld and 73.3 on AndroidWorld. Practical guidance for avoiding reward hacking.

<https://arxiv.org/abs/2509.02544>

Recommendation System Manipulation

Manipulating Recommender Systems: A Survey of Poisoning Attacks and Countermeasures

Deldjoo et al. • arXiv:2404.14942

Why: Comprehensive survey of how recommendation systems can be manipulated via data poisoning. Covers AI-based attacks using GANs and RL. Relevant to understanding detection mechanisms platforms deploy.

<https://arxiv.org/abs/2404.14942>

Adversarial Attacks and Detection on RL-Based Interactive Recommender Systems

Cao et al. • arXiv:2006.07934

Why: Shows that strategically-timed attacks achieve comparable performance with only 1/3 to 1/2 attack frequency — directly relevant to our weekly session scheduling strategy.

<https://arxiv.org/abs/2006.07934>

Recommender Systems and Algorithmic Hate

Multiple authors • arXiv:2209.02159

Why: Documents the echo chamber effect and algorithmic stickiness that our approach exploits. Provides the sociological framing for why feed manipulation persists after the agent stops.

<https://arxiv.org/abs/2209.02159>

Self-Adapting Memory

Titans: Learning to Memorize at Test Time

Behrouz, Hashemi, Srinivasan, Maddix, Wang (Google Research) • arXiv:2501.00663

Why: Introduces a learned long-term memory module for transformers that updates its own parameters at test time via gradient-based surprise signals. The memory as gate (MAG) variant is most promising for LYTENAP — it modulates attention output without disrupting the base policy. The VAGEN prediction error signal maps directly to the surprise mechanism that drives memory updates.

<https://arxiv.org/abs/2501.00663>

Surveys & Reference

A Survey on Vision-Language-Action Models for Embodied AI

Ma et al. • arXiv:2405.14093

Why: Comprehensive taxonomy of VLA architectures, training strategies, and deployment considerations.

<https://arxiv.org/abs/2405.14093>

LLM-Powered GUI Agents in Phone Automation: Surveying Progress and Prospects

Liu et al. • arXiv:2504.19838

Why: Survey of LLM-powered phone automation agents covering task architectures, SFT/RL training strategies, and open challenges.

<https://arxiv.org/abs/2504.19838>

Exit Criteria

- Start with VAGEN — this is the architectural foundation for everything we're building
- Read DigiRL next — closest prior work, validates that RL on real devices dramatically outperforms SFT
- Skim Qwen2-VL/2.5-VL for backbone architecture details you'll need during implementation
- Read MobileRL and UI-TARS-2 for practical RL training infrastructure and stability insights
- Read the recommender system papers to understand the adversarial landscape and detection risks
- Read Titans when approaching Phase 5 — understand the memory architecture before integration
- Surveys are reference material — skim for context, return to specific sections as needed